# ZX Spectrum Screen Handling

A guide for the 48K Spectrum: where the screen lives, why its layout is shuffled, and how to work out addresses quickly in Z80 code.

Every number and worked example in this guide was checked with a small Python script that implements the address formulas and simulates the Z80 routines line by line.

**Notation.** `$` means hexadecimal, as in Spectrum assemblers, so `$4000` = 16384. `%` means binary. x is the pixel column (0 to 255, left to right). y is the pixel row (0 to 191, **top to bottom**). Note that BASIC's `PLOT` counts y upwards from the bottom instead, so its y is different.

---

## Contents


1. Overview: the memory map
2. The thirds, in plain words and pictures
3. The bit layout
4. The attribute byte and attribute clash
5. Worked examples
6. Contended memory and the ULA
7. Quick reference


---

## 1. Overview: the memory map

The screen is two separate blocks of RAM, one after the other:

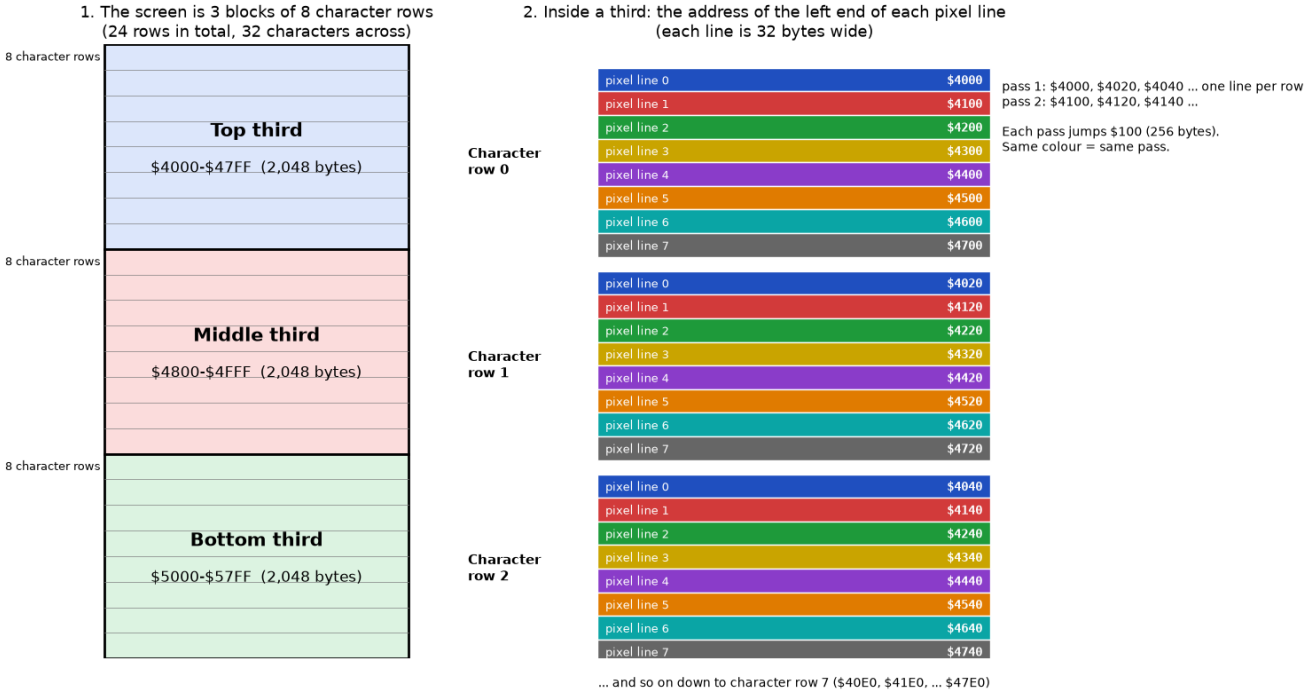
| Block | Proper name | Hex | Decimal | Size | What it holds |
|---|---|---|---|---|---|
| Pixels | **display file** | `$4000-$57FF` | 16384-22527 | 6,144 bytes | 1 bit per pixel: 1 = INK, 0 = PAPER |
| Colours | **attribute file** | `$5800-$5AFF` | 22528-23295 | 768 bytes | 1 byte per 8×8 character cell |

- The screen is **256 × 192 pixels**, which is **32 × 24 characters** of 8 × 8 pixels.
- Each display-file byte holds 8 pixels side by side. 256 pixels ÷ 8 = **32 bytes per pixel line**, and 32 × 192 = 6,144 bytes.
- The attribute file has one byte per character cell: 32 × 24 = 768 bytes.
- Together they make 6,144 + 768 = **6,912 bytes**. That is exactly the length of a `SCREEN$` file, because `SAVE "name" SCREEN$` is just `SAVE "name" CODE 16384,6912`.
- Inside a byte, the **leftmost pixel is bit 7** and the rightmost is bit 0. Pixel x is bit (7 − x mod 8) of its byte.

The attribute file is laid out the simple way: left to right, top to bottom. The display file is the one with the famous shuffle, so most of this guide is about the display file.

---

# 2. The thirds, in plain words and pictures



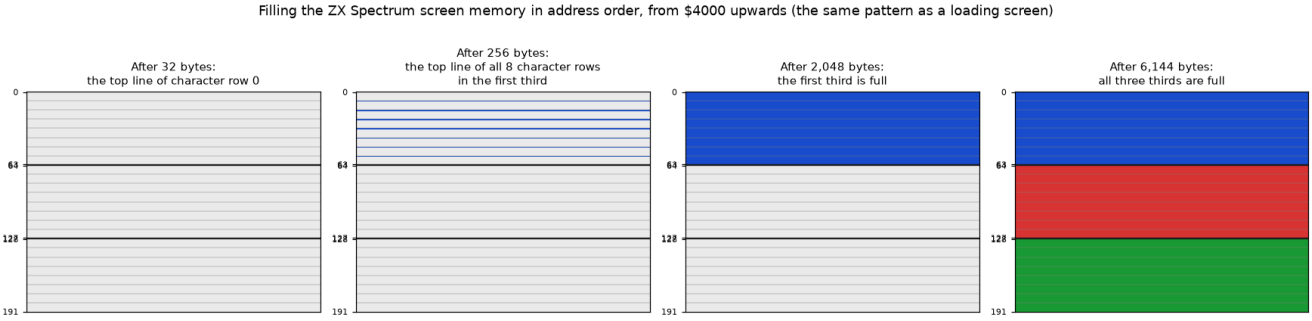

**Picture 1 (left):** the screen is three blocks stacked on top of each other. Each block is 8 character rows tall (64 pixel lines) and takes exactly 2,048 bytes:

| Third | Character rows | Pixel lines (y) | Display file |
|---|---|---|---|
| Top | 0-7 | 0-63 | `$4000-$47FF` |
| Middle | 8-15 | 64-127 | `$4800-$4FFF` |
| Bottom | 16-23 | 128-191 | `$5000-$57FF` |

**Picture 2 (right):** inside a third, memory does not run straight down the screen. It goes like this:

1. The first 32 bytes (`$4000-$401F`) are the **top pixel line of character row 0**.
2. The next 32 bytes (`$4020`) are the **top pixel line of character row 1**, 8 pixels further down. Then row 2 (`$4040`), and so on to row 7 (`$40E0`). That is one pass of 256 bytes: the top line of all 8 character rows.
3. The next pass (`$4100` onwards) goes back up and fills the **second pixel line** of every character row. Then the third line (`$4200`), and so on, 8 passes in all.
4. After 8 passes (8 × 256 = 2,048 bytes) the third is full, and the next third starts.

## You have seen this happen



A loading screen loads its bytes in address order, from `$4000` upwards. That is why it appears as thin lines spaced 8 pixels apart, which fill in until the top third is solid, then the middle third, then the bottom. After that the colours arrive all at once, because the attribute file comes last.

### Why it is useful

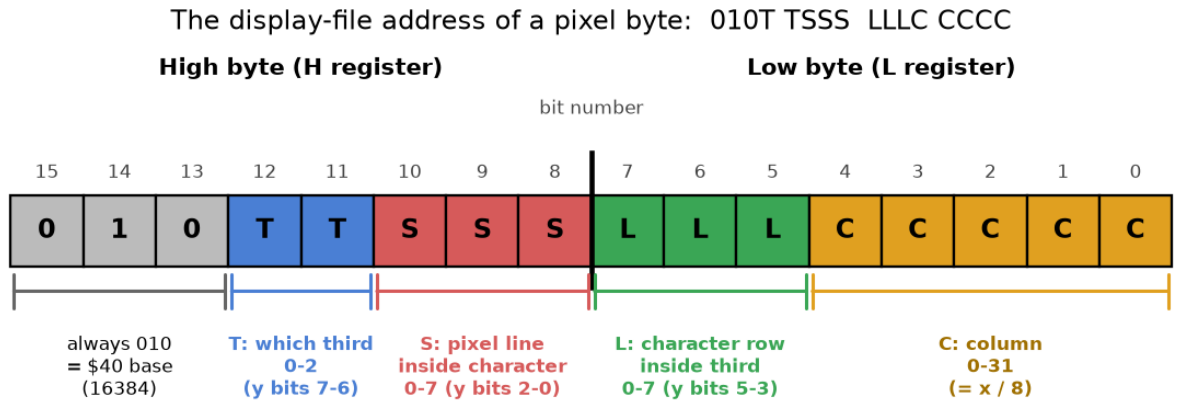
Look down the right-hand column of picture 2 for character row 0: `$4000`, `$4100`, `$4200` … `$4700`. Moving **down one pixel line inside a character adds 1 to the high byte** of the address, which is `INC H` in Z80 (4 T-states). Drawing a character or a sprite column is "write a byte, `INC H`" eight times. With a plain top-to-bottom layout you would need "add 32" on every line, with carries.

The price is that crossing from the bottom of one character row into the next, or from one third into the next, needs a little extra arithmetic. Section 5 shows exactly what.

---

## 3. The bit layout

Now the same thing in bits. The 16-bit display-file address of the byte holding pixel (x, y) is:



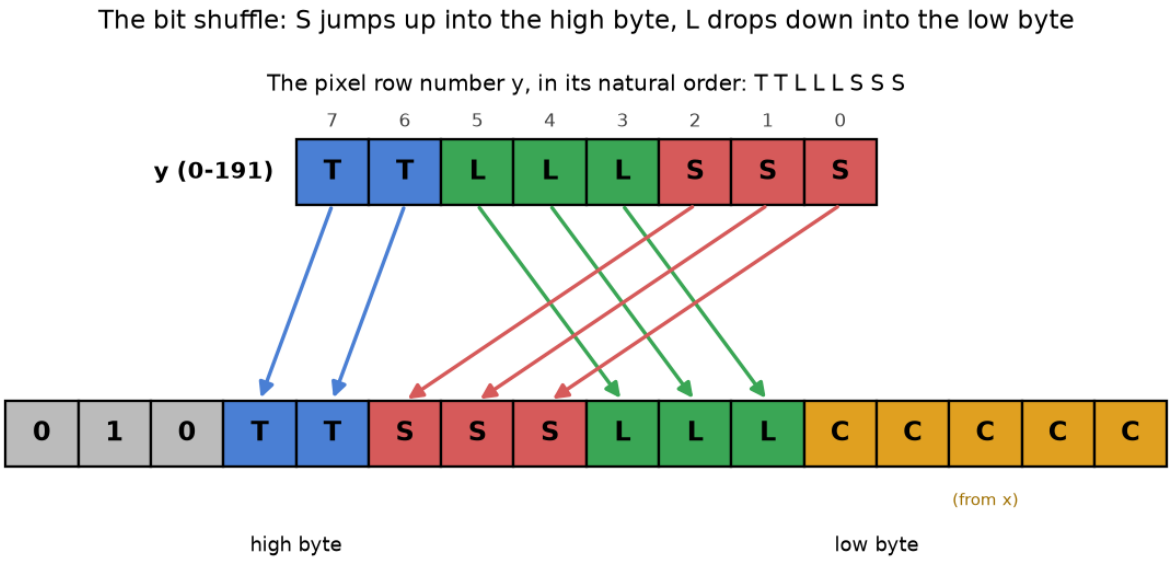

```
      high byte (H)          low byte (L)
    0 1 0 T T S S S       L L L C C C C C
```

| Letters | Bits | Meaning | Range | Comes from |
|---|---|---|---|---|
| `010` | 15-13 | fixed, makes the base address `$4000` | - | - |
| TT | 12-11 | **third** | 0-2 | y ÷ 64 (y bits 7-6) |
| SSS | 10-8 | **pixel line inside the character** | 0-7 | y mod 8 (y bits 2-0) |
| LLL | 7-5 | **character row inside the third** | 0-7 | (y ÷ 8) mod 8 (y bits 5-3) |
| CCCCC | 4-0 | **column** (byte across the line) | 0-31 | x ÷ 8 |

(All divisions here are whole-number divisions, throwing away the remainder.)

### y is shuffled

Written in binary, y (0 to 191) naturally splits as `TT LLL SSS`: which third, which character row in that third, which line in that character. The address uses the same three pieces, but **swaps the order of `LLL` and `SSS`**:




```
y       =       T T L L L S S S
address  = 0 1 0  T T S S S   L L L C C C C C
```

Because SSS lands in the bottom 3 bits of H, adding 1 to H moves down one pixel line. Because LLL lands in the top 3 bits of L, adding 32 (`%00100000`) to L moves down one character row.

The same formula in decimal:

```
address = 16384 + 2048 × third + 256 × line_in_char + 32 × row_in_third + column
```

## The attribute address

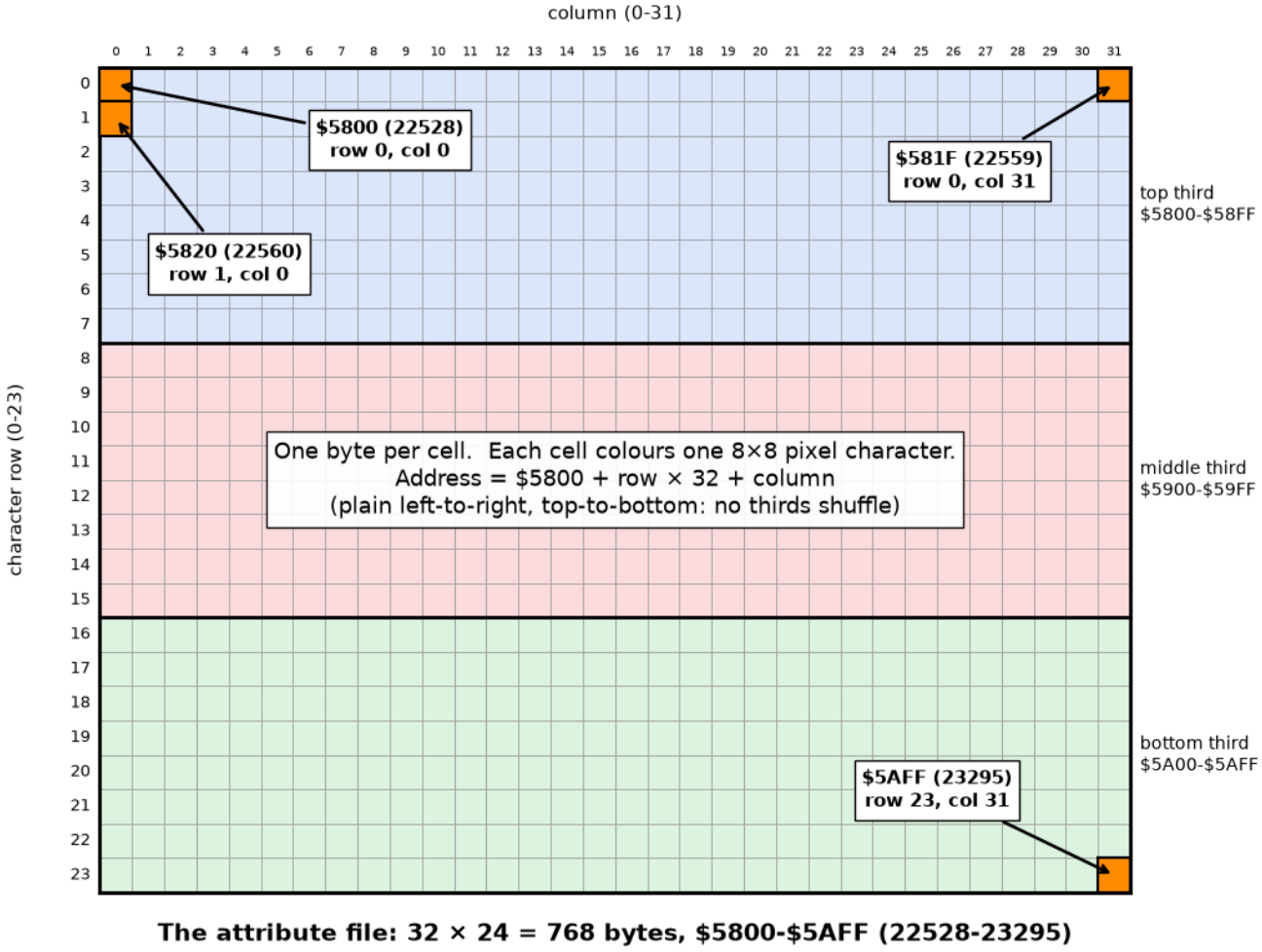
The attribute file is in plain order, so:

```
attribute address = $5800 + row × 32 + column        (row 0-23, column 0-31)
                  = 22528 + row × 32 + column
```

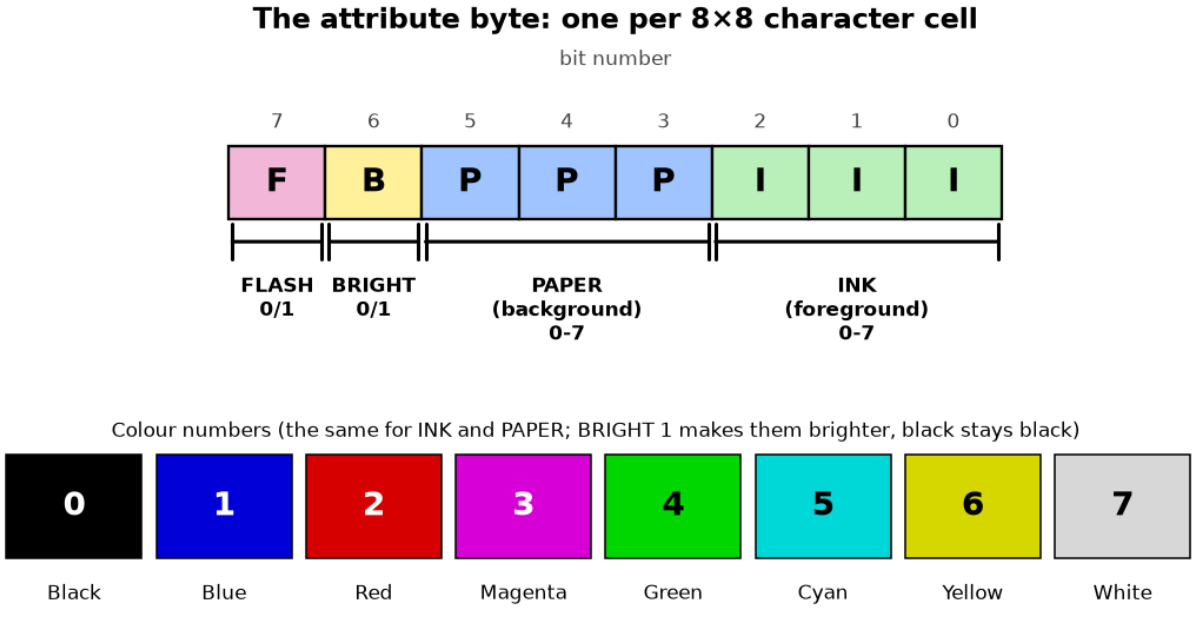
In bits, with the character row (0-23) written as TT LLL (third, then row inside the third):

```
attribute:   0 1 0 1 1 0 T T   L L L C C C C C
display:     0 1 0 T T S S S   L L L C C C C C
```

The **low byte is identical** in both. Only the high byte changes: for the attribute it is `$58 + third`, so `$58`, `$59` or `$5A`. That makes converting a display address to its attribute address very cheap (section 5.6).




## 4. The attribute byte and attribute clash




Each attribute byte colours one 8 × 8 cell:

| Bit | 7 | 6 | 5-3 | 2-0 |
|---|---|---|---|---|
| Field | FLASH | BRIGHT | PAPER (colour of 0 bits) | INK (colour of 1 bits) |

```
attribute = 128 × FLASH + 64 × BRIGHT + 8 × PAPER + INK
```

Colour numbers, for both INK and PAPER: 0 black, 1 blue, 2 red, 3 magenta, 4 green, 5 cyan, 6 yellow, 7 white.

- **BRIGHT 1** makes both colours in the cell brighter (black stays black).
- **FLASH 1** makes the ULA swap INK and PAPER every 16 frames (about 0.32 seconds).

Examples (checked by the script):

| Wanted | Calculation | Byte |
|---|---|---|
| Black ink on white paper (the power-on default) | 8 × 7 + 0 | 56 = $38 |
| Yellow ink on blue paper | 8 × 1 + 6 | 14 = $0E |
| Bright white ink on blue paper | 64 + 8 × 1 + 7 | 79 = $4F |
| Flashing yellow ink on red paper | 128 + 8 × 2 + 6 | 150 = $96 |

### Attribute clash

Pixels are stored one bit each, but colour is stored once per 8 × 8 cell. So **each cell can show only two colours, its INK and its PAPER**. When a sprite is drawn, it only changes pixel bits. When the sprite moves into a cell, there are two choices, and both show:

1. **Leave the cell's attribute alone.** The sprite's pixels take on that cell's INK colour, so a white sprite turns green as it passes over a cell of green seaweed.
2. **Write the sprite's own attribute into the cell.** Now the sprite keeps its colour, but so does everything else in that cell, so the surrounding 8 × 8 block of background suddenly changes colour too.

That is attribute clash. In Atlantis you can see choice 1 deliberately: a plant's spare cell has blue paper, so it looks like water until a sprite swims into it and its pixels show as the cell's green ink. Common ways to live with clash are monochrome play areas, sprites that take the background colour, and designing scenery so colours change on 8-pixel boundaries.

---

## 5. Worked examples

### 5.1 The address of pixel (x, y)

Method: split y into `TT LLL SSS`, work out the column x ÷ 8, then put the pieces back together as `010TTSSS LLLCCCCC`.

**Example A: pixel (0, 0)**, the top-left corner.

- y = 0 = `%00 000 000`, so third 0, row-in-third 0, line 0. Column = 0 ÷ 8 = 0.
- H = `%010 00 000` = `$40`, L = `%000 00000` = `$00`.
- **Address `$4000` = 16384**, bit 7.

**Example B: pixel (100, 50)**

- y = 50 = `%00 110 010`, so third **0**, row-in-third **6**, line **2**. (Check: 50 = 0 × 64 + 6 × 8 + 2.)

- Column = 100 ÷ 8 = **12** remainder 4, so the pixel is bit 7 − 4 = **bit 3**.
- H = `%010 00 010` = `$42`.
- L = `%110 01100` = `$CC`.
- **Address `$42CC` = 17100.**
- Decimal check: 16384 + 2048 × 0 + 256 × 2 + 32 × 6 + 12 = 16384 + 512 + 192 + 12 = **17100**.

**Example C: pixel (255, 191)**, the bottom-right corner.

- y = 191 = `%10 111 111`, so third **2**, row-in-third **7**, line **7**.
- Column = 255 ÷ 8 = **31** remainder 7, so **bit 0**.
- H = `%010 10 111` = `$57`, L = `%111 11111` = `$FF`.
- **Address `$57FF` = 22527**, the very last byte of the display file.
- Decimal check: 16384 + 4096 + 1792 + 224 + 31 = **22527**.

**Example D: pixel (128, 77)**, in the middle third.

- y = 77 = `%01 001 101`, so third **1**, row-in-third **1**, line **5**. (Check: 77 = 64 + 8 + 5. That is character row 8 + 1 = 9 of the screen.)
- Column = 128 ÷ 8 = **16** remainder 0, so **bit 7**.
- H = `%010 01 101` = `$4D`, L = `%001 10000` = `$30`.
- **Address `$4D30` = 19760.**
- Decimal check: 16384 + 2048 + 1280 + 32 + 16 = **19760**.

## 5.2 Moving down one pixel line inside a character: `INC H`

From pixel (100, 50) at `$42CC` (line 2 of its character), the pixel below, (100, 51), is line 3. Only `SSS` changes, from 2 to 3, and `SSS` is the bottom of H:

```
$42CC  ->  INC H  ->  $43CC
```

This works for lines 0→1, 1→2 ... 6→7 of any character. It goes wrong only when moving from line 7 to the next character row.

## 5.3 Crossing into the next character row in the same third

Take pixel (40, 15) → (40, 16): the bottom line of character row 1 to the top line of character row 2.

- (40, 15): y = 15 = `%00 001 111`. H = `%010 00 111` = `$47`, L = `%001 00101` = `$25`. Address **`$4725`**.
- (40, 16): y = 16 = `%00 010 000`. H = `$40`, L = `%010 00101` = `$45`. Address **`$4045`**.

What `INC H` alone gives: `$4725` → `$4825`. The `SSS` bits overflowed from 7 to 0 and carried 1 into `TT`, so this is the middle third, which is wrong. Two fixes are needed:

1. **L + 32:** move down one character row. `$25` + `$20` = `$45`. No carry out of L, so we are still in the same third.
2. **H − 8:** undo the carry into `TT` that `INC H` caused. `$48` − 8 = `$40`.

Result: **`$4045`**, which is correct. Rule: when `INC H` makes the bottom 3 bits of H zero, add 32 to L; if that does **not** carry, also subtract 8 from H.

## 5.4 Crossing from one third into the next: y = 63 → y = 64

- (0, 63): y = 63 = `%00 111 111`. Address **`$47E0`** (line 7 of row 7, the last pixel line of the top third).
- (0, 64): y = 64 = `%01 000 000`. Address **`$4800`** (the first line of the middle third).

Step by step from `$47E0`:

1. `INC H`: `$47` → `$48`. The bottom 3 bits of H are now 0, so we have left the character.
2. L + 32: `$E0` + `$20` = `$100`, so L = `$00` **with carry**. The carry means `LLL` wrapped from 7 back to 0, so we really have moved into the next third.
3. Because there was a carry, **do not** subtract 8: the carry into `TT` from step 1 is exactly what we want.

Result: **`$4800`**, correct.

## 5.5 Character (row, column) to display address and attribute address

Take character **row 10, column 5**.

Display address (pixel line 0 of that character):

- Row 10 = `%01 010`, so third = 10 ÷ 8 = **1**, row-in-third = 10 mod 8 = **2**.
- H = `%010 01 000` = `$48`. L = `%010 00101` = `$45`.
- **`$4845` = 18501.** Decimal check: 16384 + 2048 × 1 + 32 × 2 + 5 = **18501**.
- The other 7 pixel lines of this character are `$4945`, `$4A45` … `$4F45` (`INC H` each time).

Attribute address:

- `$5800` + 10 × 32 + 5 = `$5800` + 325 = `$5800` + `$145` = **`$5945` = 22853.**
- Decimal check: 22528 + 320 + 5 = **22853**.
- Note the low byte `$45` is the same as the display address, and the high byte is `$58` + third 1 = `$59`.

## 5.6 Display address to attribute address in Z80

Given HL pointing anywhere in the display file, this makes HL point at that cell's attribute. L is not touched.

```
                ; HL = display address, H = %010TTSSS       T-states
LD   A,H        ; A = %010TTSSS                              4
RRCA            ; A = %S010TTSS   rotate right, bit 0 -> bit 7  4
RRCA            ; A = %SS010TTS                              4
RRCA            ; A = %SSS010TT   TT is now in bits 1-0      4
AND  3          ; A = %000000TT   keep only the third         7
OR   $58        ; A = %010110TT   = $58, $59 or $5A           7
LD   H,A        ; HL = attribute address                     4
                ;                                   total:  34
```

Worked through for `$4845` (row 10, column 5): H = `$48` = `%01001000` → `%00100100` → `%00010010` → `%00001001` → AND 3 = `%01` → OR `$58` = `$59`. HL = **`$5945`**, matching 5.5. The script ran this sequence for all 6,144 display bytes and every one gave the right attribute address.

## 5.7 "Next line down": a general routine

This moves HL from any display address to the byte directly below it, handling both boundaries. T-states are standard Z80 timings, **uncontended** (see section 6). Where a conditional `RET` is used, the first figure is when it returns, the second when it does not.

```
; NEXT_LINE_DOWN
; In:  HL = display-file address on pixel line y (0-190)
; Out: HL = address of the same column on line y+1
; Uses A, flags.                                          T-states
NextLineDown:
    INC  H          ; SSS + 1: down one pixel line            4
    LD   A,H        ;                                         4
    AND  7          ; are the line bits (SSS) now 0?          7
    RET  NZ         ; no: still inside the character, done  11 / 5
                    ; yes: we came off the bottom of a character,
                    ; and INC H has carried 1 into the third bits.
    LD   A,L        ;                                         4
    ADD  A,32       ; LLL + 1: down one character row         7
    LD   L,A        ;                                         4
    RET  C          ; carry: LLL went 7 -> 0, so we really did
                    ; enter the next third and H is right, done  11 / 5
    LD   A,H        ; no carry: still in the same third,      4
    SUB  8          ; so undo the carry into the third bits   7
    LD   H,A        ;                                         4
    RET             ;                                        10
```

Cost of each path, including the final `RET` (add 17 for the `CALL`):

| Case | How often in 191 moves | T-states |
|---|---|---|
| Inside a character | 168 | 26 |
| Into the next third (y = 63→64, 127→128) | 2 | 46 |
| Into the next character row, same third | 21 | 65 |

The script simulated this routine for every line 0-190 and every column, and checked the result against the formula. Do not call it from line 191: it returns `$58xx`, which is the attribute file.

## 5.8 Why games use a row address table

Working out an address from y with shifts and masks costs several dozen T-states, and the next-line routine above has three different timings. Games that draw sprites at any height usually **precompute the address of the start of every pixel line once**, at start-up:

- 192 lines × 2 bytes = **384 bytes** of table.
- Each entry is the address of column 0 on that line, such as `$4000`, `$4100` … `$4700`, `$4020` …
- Every entry's low byte has the column bits clear, so the column can be added with a plain `OR`.

A simple lookup, with y in A and the column (x ÷ 8) in C:

```
                    ;                                         T-states
    LD   L,A        ; HL = y                                  4
    LD   H,0        ;                                         7
    ADD  HL,HL      ; HL = y × 2 (2 bytes per entry)          11
    LD   DE,RowTable ;                                       10
    ADD  HL,DE      ; HL -> entry for line y                 11
    LD   A,(HL)     ; low byte of line start                  7
    INC  HL         ;                                         6
    LD   H,(HL)     ; high byte of line start                 7
    OR   C          ; add the column (bits 4-0 were 0)        4
    LD   L,A        ; HL = display address of (x, y)          4
                    ;                                  total: 71
```

The real benefit is in a sprite loop: stepping a pointer through the table gives the next line's address at the same small cost on **every** line, with no boundary tests. Splitting the table into separate low-byte and high-byte halves, aligned to 256-byte pages, makes it faster still. Our engine's sprite code uses a table like this, which is why its sprites can sit at any pixel height.

### 5.9 The ROM's own routine

The 48K ROM has a pixel-address routine, **PIXEL-ADD at `$22AA`** (8874), used by `PLOT` and `POINT` (SkoolKid's ROM disassembly). It takes B = y and C = x in **BASIC coordinates**: y counts up from the bottom and must be 0-175 (it works on 175 − y, so the bottom two character rows can't be reached). It returns HL = display address and A = x mod 8. It is handy to read as an example of the bit shuffle, but games normally use their own top-down coordinates and a table.

---

## 6. Contended memory and the ULA

- On a 48K Spectrum, `$4000-$7FFF` is **contended memory**: the ULA shares it with the Z80. The screen is in this block, so **every read or write of screen memory is contended**.
- While the ULA draws the 192 picture lines, it fetches screen bytes during 128 T-states of each 224 T-state line. In each group of 8 T-states it reads a pixel byte, its attribute byte, the next pixel byte, and the next attribute byte. A Z80 access to contended memory in that window can be held up by up to 6 T-states.
- During the top and bottom borders and the horizontal retrace, there is no contention, so the T-state counts above are exact then. During the picture they are minimums.
- The ULA **re-reads the attribute byte on each of the 8 pixel lines of a character row**. So if you change an attribute between the beam drawing one pixel line and the next, the colour changes part-way down the cell. That is how "multicolour" engines give 8 × 1 colour cells, by racing the beam.
- Practical rules: keep code, stack and tables above `$8000`, do as much screen drawing as possible while the beam is in the border, and draw each part of the screen after the beam has passed it, so sprites don't flicker or tear.

---

## 7. Quick reference

```
SCREEN MEMORY (48K)
  Display file   $4000-$57FF  16384-22527  6,144 bytes  1 bit per pixel
  Attribute file $5800-$5AFF  22528-23295    768 bytes  1 byte per 8x8 cell
  Total 6,912 bytes = SCREEN$.   256 x 192 pixels = 32 x 24 characters
  Leftmost pixel of a byte = bit 7.  Pixel x is bit (7 - x mod 8).

  Thirds: top $4000-$47FF (y 0-63), middle $4800-$4FFF (64-127),
          bottom $5000-$57FF (128-191)

DISPLAY ADDRESS            y = T T L L L S S S
  010T TSSS LLLC CCCC      T = third (0-2)       S = line in char (0-7)
                           L = row in third (0-7) C = column = x/8 (0-31)
  = 16384 + 2048*T + 256*S + 32*L + C

ATTRIBUTE ADDRESS          0101 10TT LLLC CCCC
  = $5800 + 32*row + column  (row 0-23)   Low byte = display low byte

ATTRIBUTE BYTE             F B P P P I I I
  = 128*FLASH + 64*BRIGHT + 8*PAPER + INK
  0 black 1 blue 2 red 3 magenta 4 green 5 cyan 6 yellow 7 white

MOVING AROUND
  Right one byte (8 px)          INC L    (within a line)
  Down 1 line inside a char      INC H
  Down 1 char row, same third    L + 32, then H - 8 (after the INC H)
  Down into next third           L + 32 carries: keep H as INC H left it
  Down 1 char row from line 0    L + 32; if carry, H + 8

DISPLAY -> ATTRIBUTE (34 T)    LD A,H / RRCA / RRCA / RRCA / AND 3 / OR $58 / LD H,A

EXAMPLES
  (0,0) $4000   (100,50) $42CC   (128,77) $4D30   (255,191) $57FF
  y 63->64: $47E0 -> $4800       (40,15) $4725 -> (40,16) $4045
  Char row 10, col 5: display $4845 (18501), attribute $5945 (22853)

ROW TABLE: 192 entries x 2 bytes = 384 bytes
ROM: PIXEL-ADD $22AA (BASIC coordinates, y 0-175 from the bottom)
CONTENDED: $4000-$7FFF, so all screen access is contended during the picture
```